Security Intergration into CI/CD pipeline

Overview:

This project is about integrating security tools and automating them into the continous integration and continous development of a software pipeline using Gihub actions so you don't have to manually scan the software each time there is a new code or feature pushed into the pipeline.

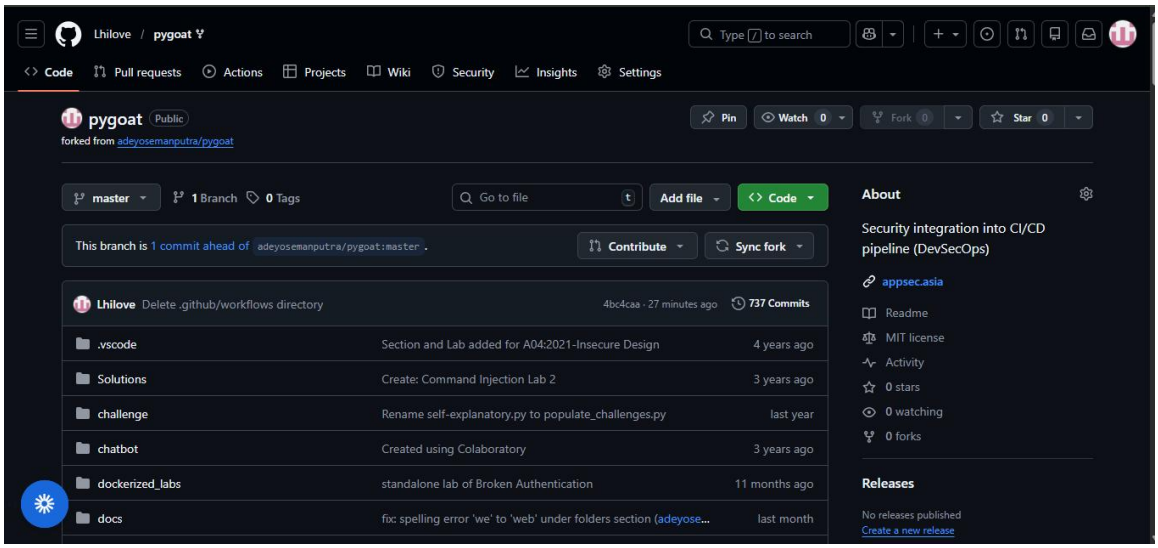
Objectives:

- To integrate a Static Analysis Security Tool (SAST).
- Build a docker image.
- To integrate an Image Scanning tool.

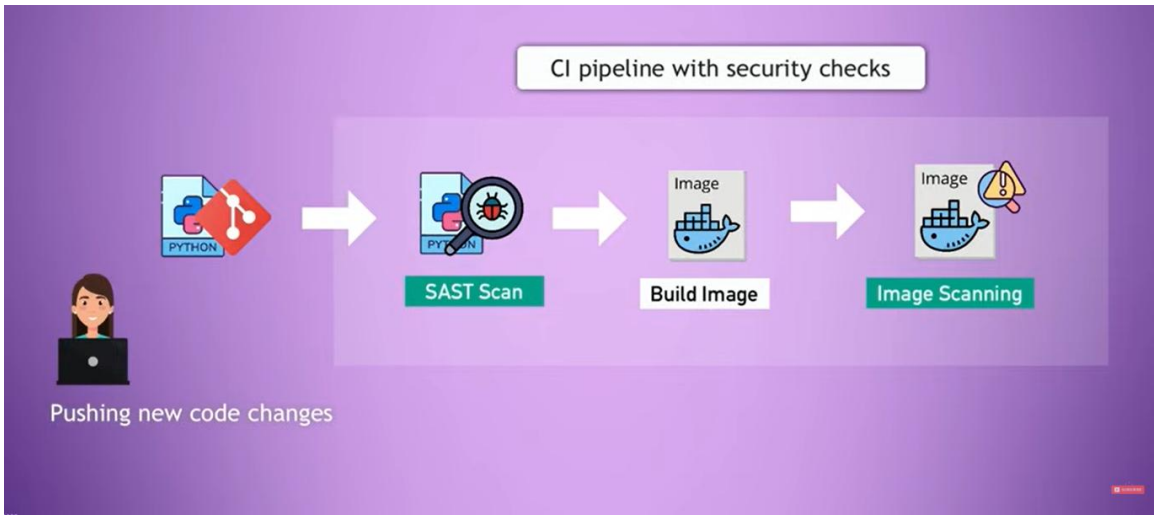
For this project, I will be using OWASP open-source project named **pygoa**t. Pygoat is a python web-app made to be vulnerable intentionally for developers, security testers to get hands-on skill.
https://owasp.org/www-project-pygoat/

STEPS:

1. Copy the project to github and understanding the DevSecOps pipeline for this project.

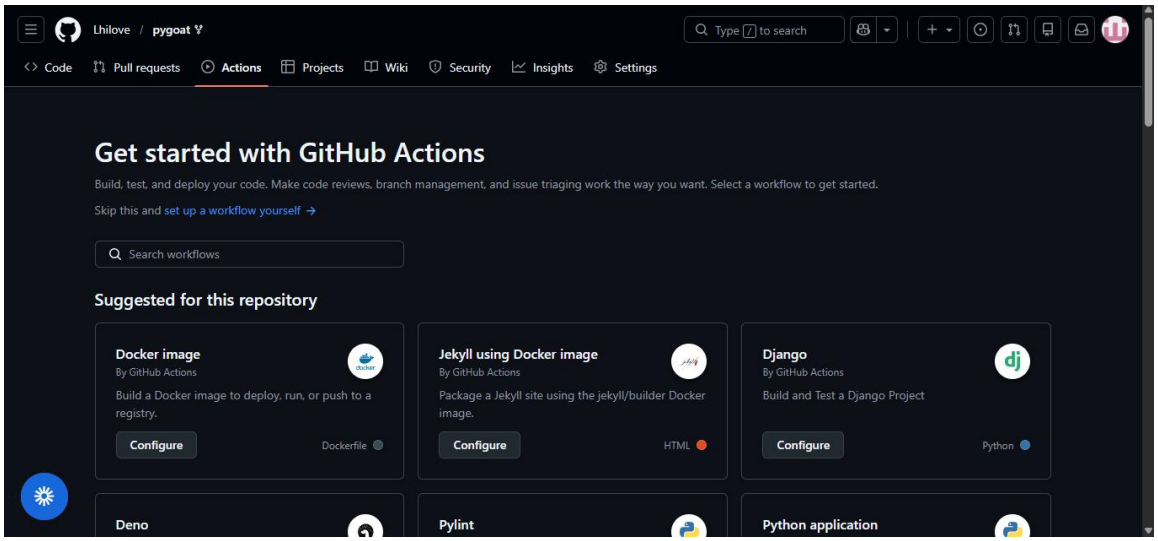


This is the project forked to my github

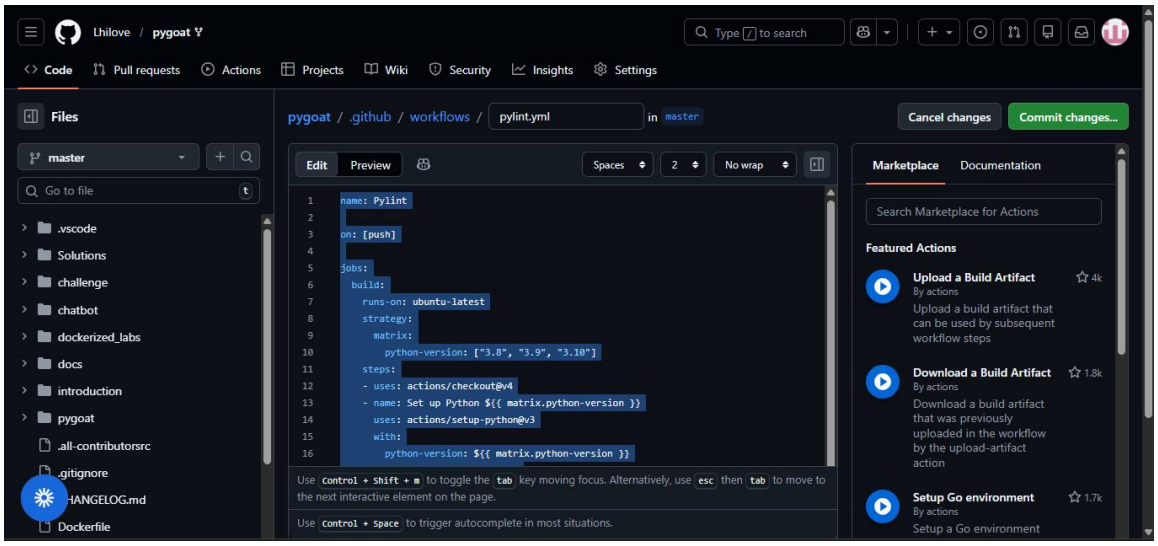


The DevSecOps CI pipeline

2. Click on Github Actions

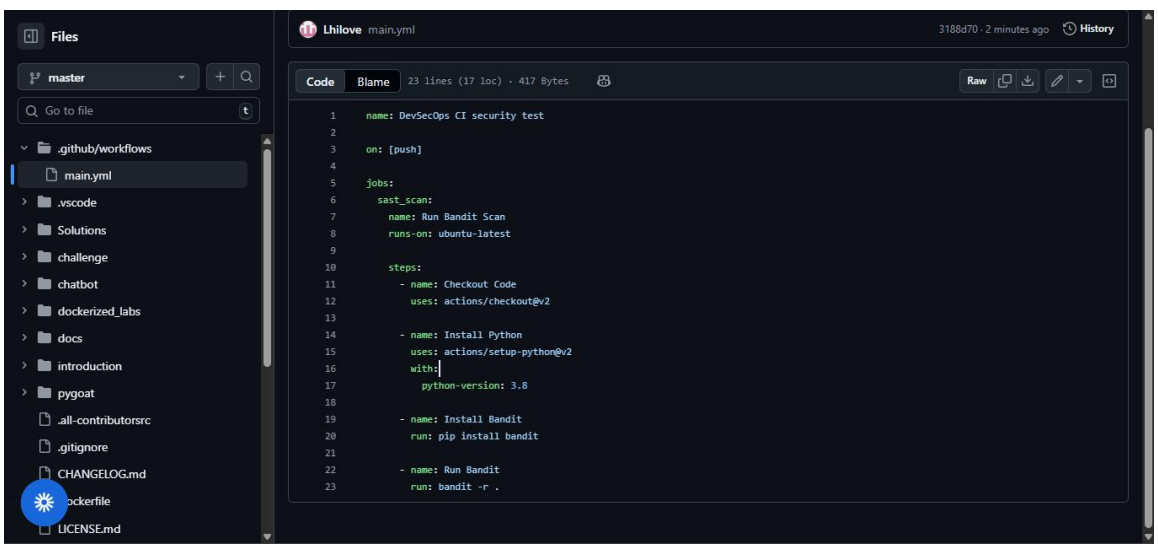


Select any of the actions



Delete the commands so you can write your own commands.

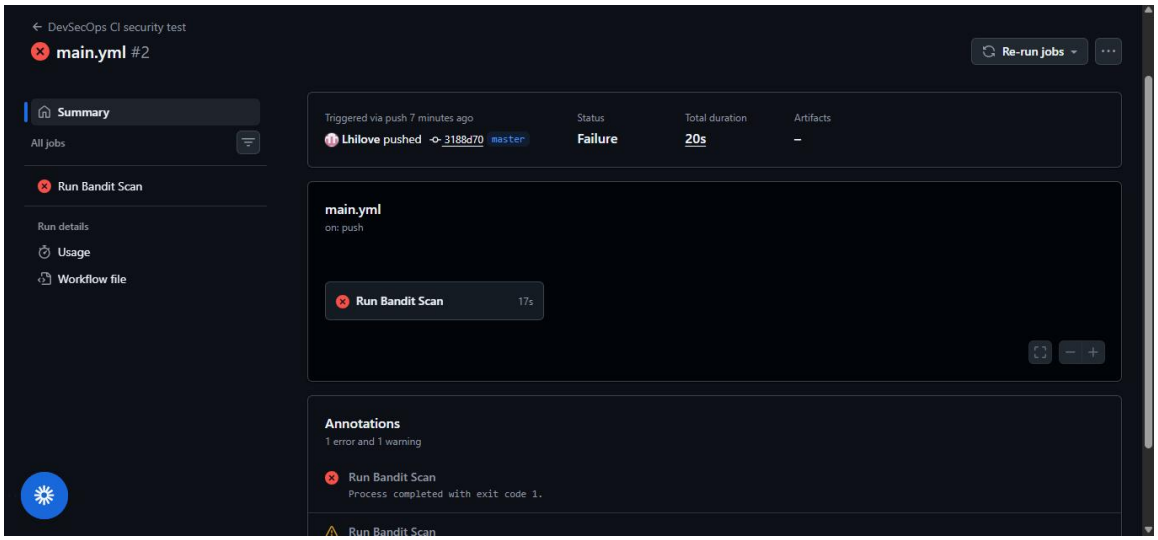
3. Integrating the SAST command, since the project I'm using was built on Python programming languages the SAST tool I will be using is called **Bandit**, This tool is the most popular scan tool which happens to be a python package.

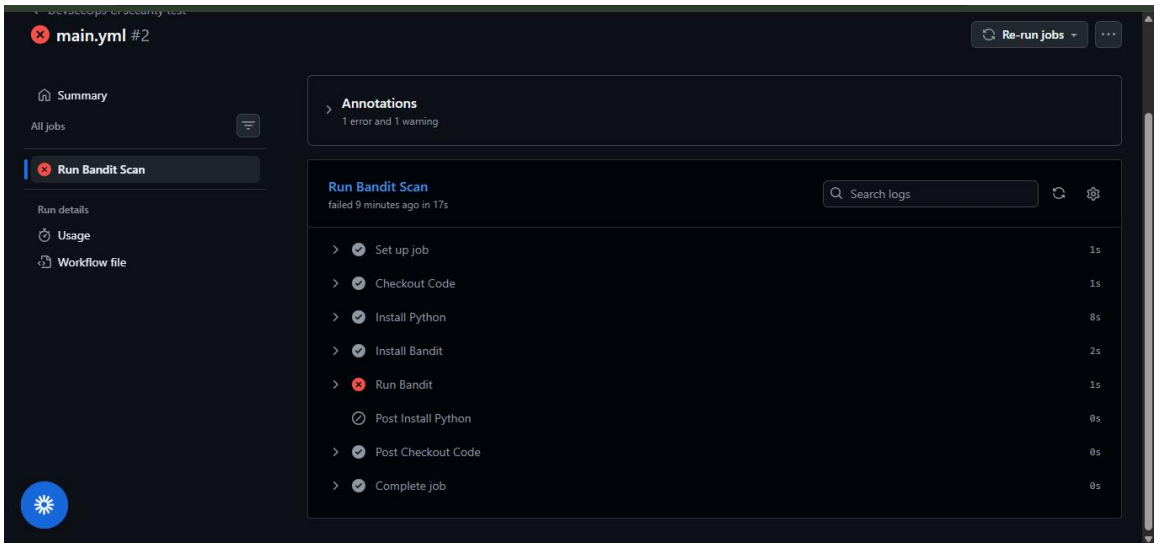


This is how to command looks like, what this command does is;
Name: name of the project (optional)
On: is the github event that triggers the workflow
Jobs: the parts that get executed whenever these events happens
Steps: these are sequence of tasks that are needed to be tested

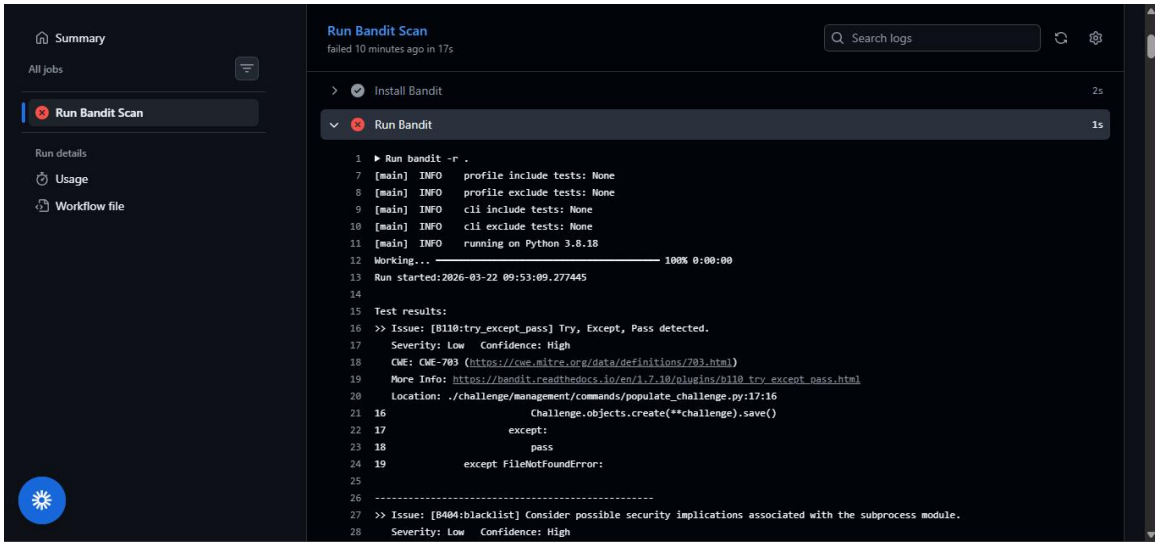
Uses: select an action
Run: runs the command line command

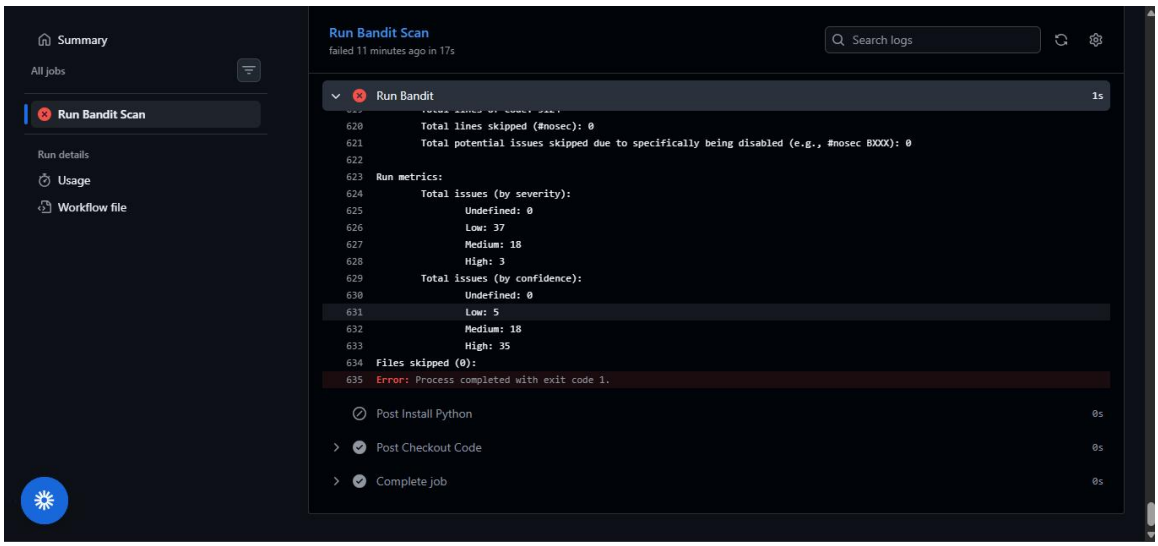
4. Commit changes and test for results: go to actions and it appears like the screenshot below, the action failed which shows that it found some vulnerabilities





Each steps was completed step by step till bandit ran.





Summary of the SAST test, the severity level and the confidence level is so important,

Severity level:

Low; refers to the impacts of the vulnerabilty found is minimal to no effects

Medium: refers to the impacts of the vulnerabilty found is pretty much something that can't be overlooked

High; refers to the impacts of the vulnerabilty found needs to be fixed quikly and urgently

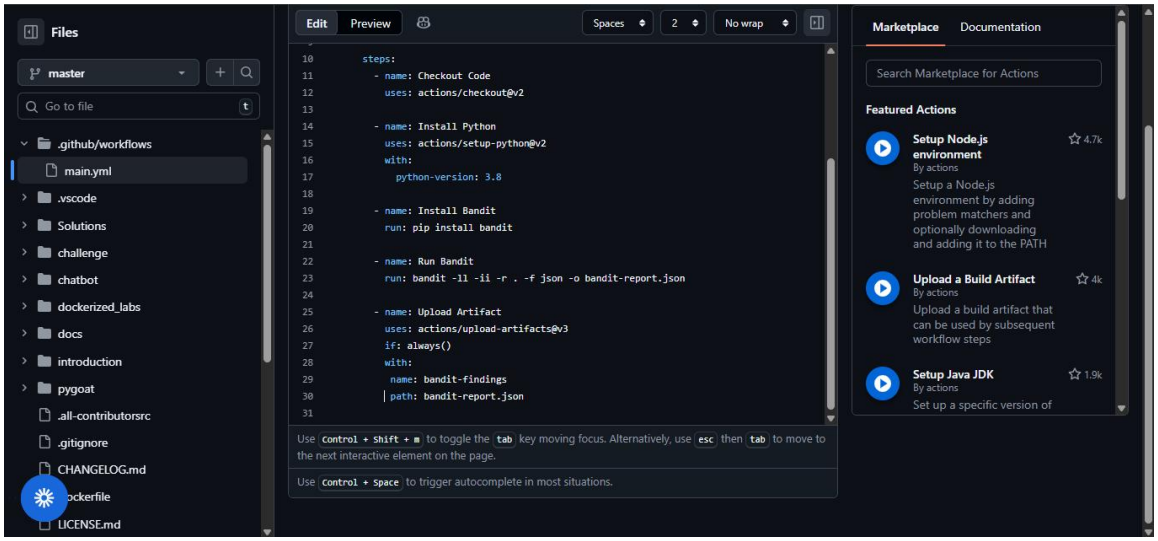
Confidence level: refers to how sure the tool is about the vulnerability

Low; Not very sure

Medium: Sure
High: Very sure

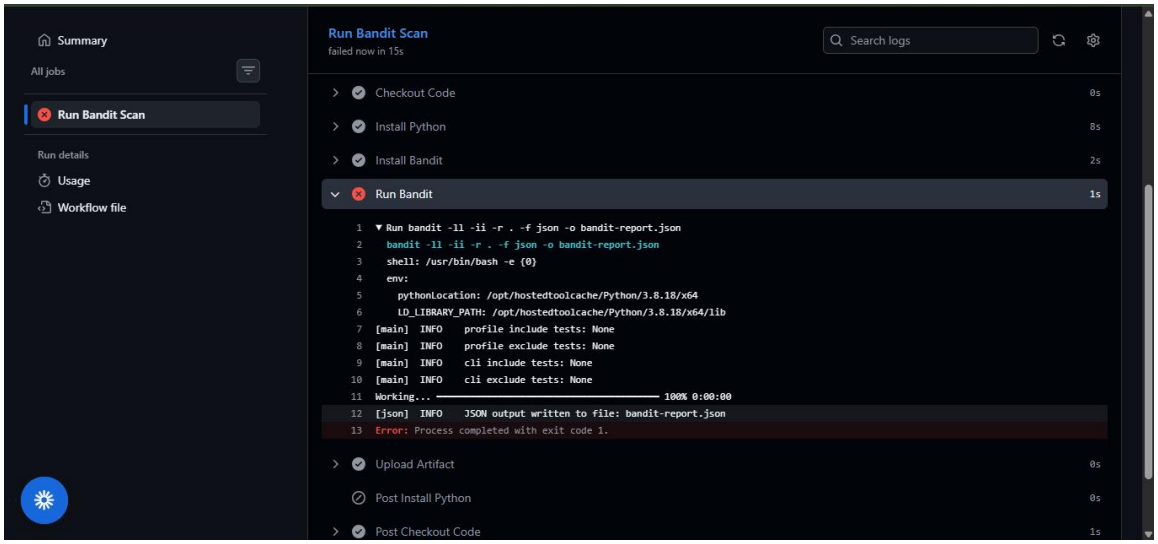
Now, I told the tool to only detect medium & high vulnerabilties so the developers can focus on the important stuffs that needs remediationand also reduce workload. Also tell bandit to make a file and make availabe to upload for Vulnerabilty Management Tool like Defect Dojo, which allows these vulnerabilites to be read in a better UI and well managed.

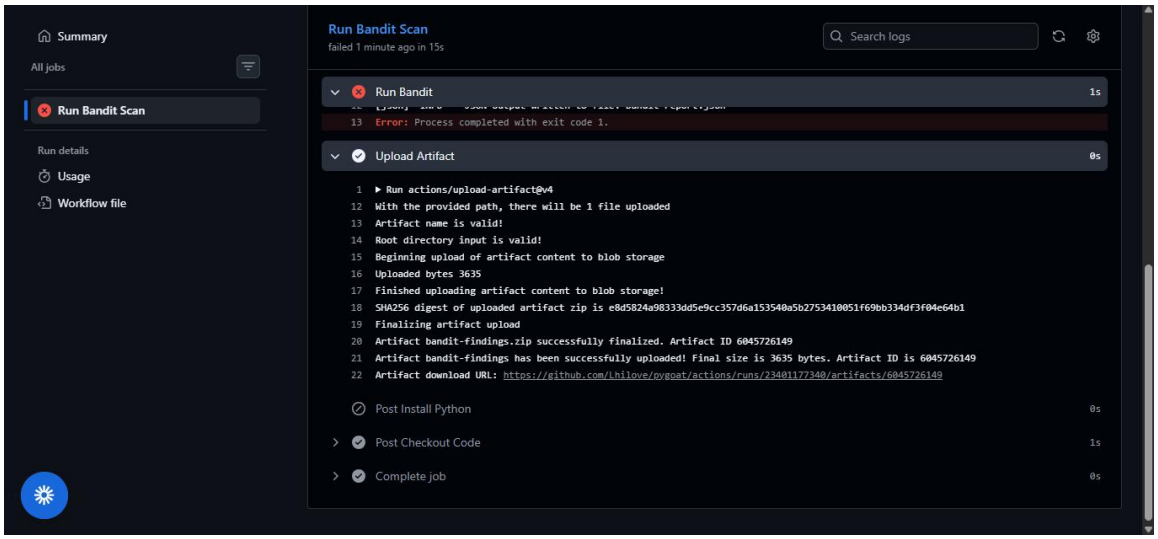
Using the bandit commands from the bandit documentation (https://bandit.readthedocs.io/en/latest/)



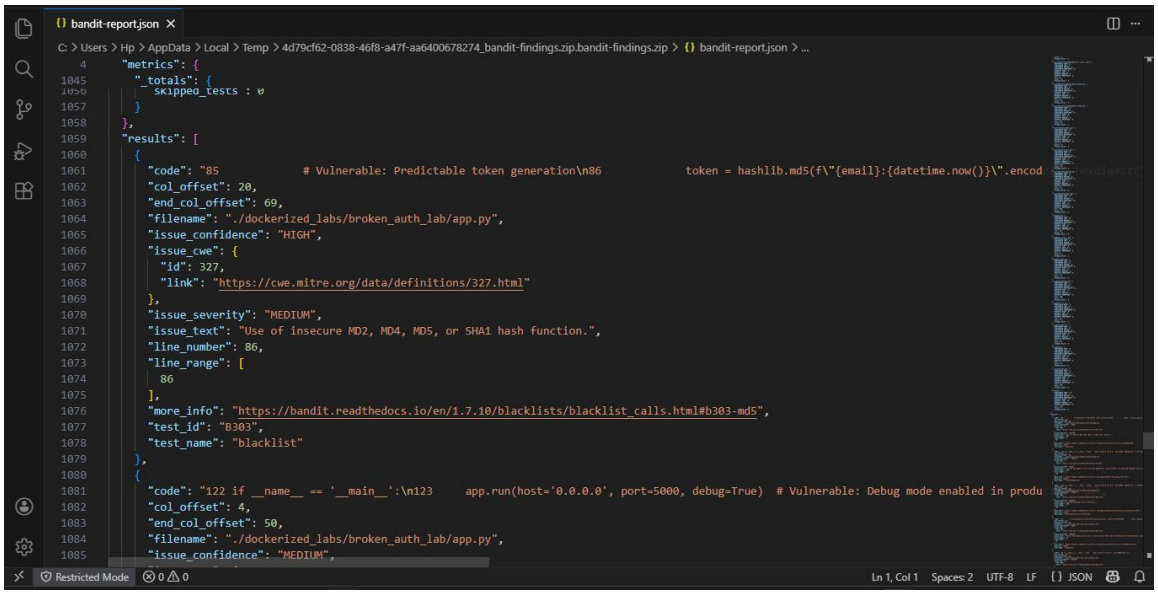
-ll: is to scan for only medium and high severity level
-ii: is to scan for medium and high confidence level
-f json: is the file format I am telling to make the report
-o: output path.

Commit changes and check for results



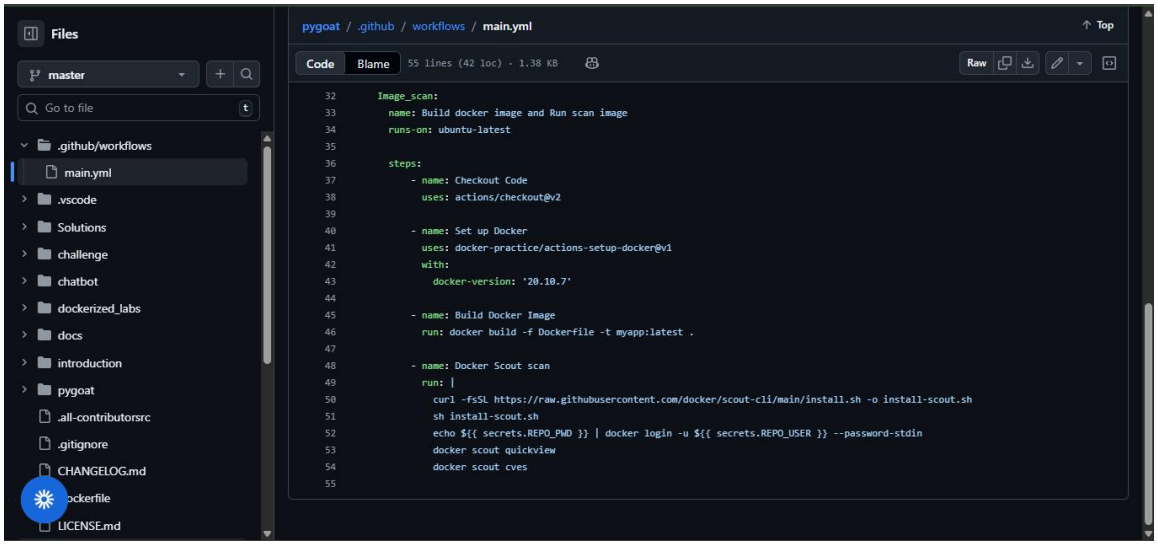


The link to the json bandit report is found under Artifact.

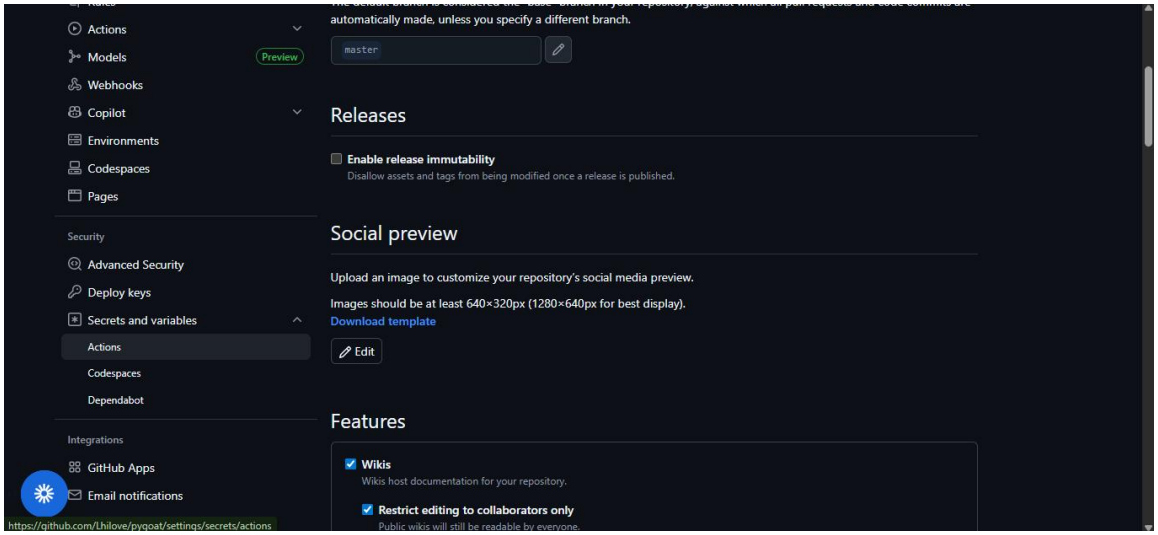


The report only generated severity level; high & medium also confidence level: high and medium.

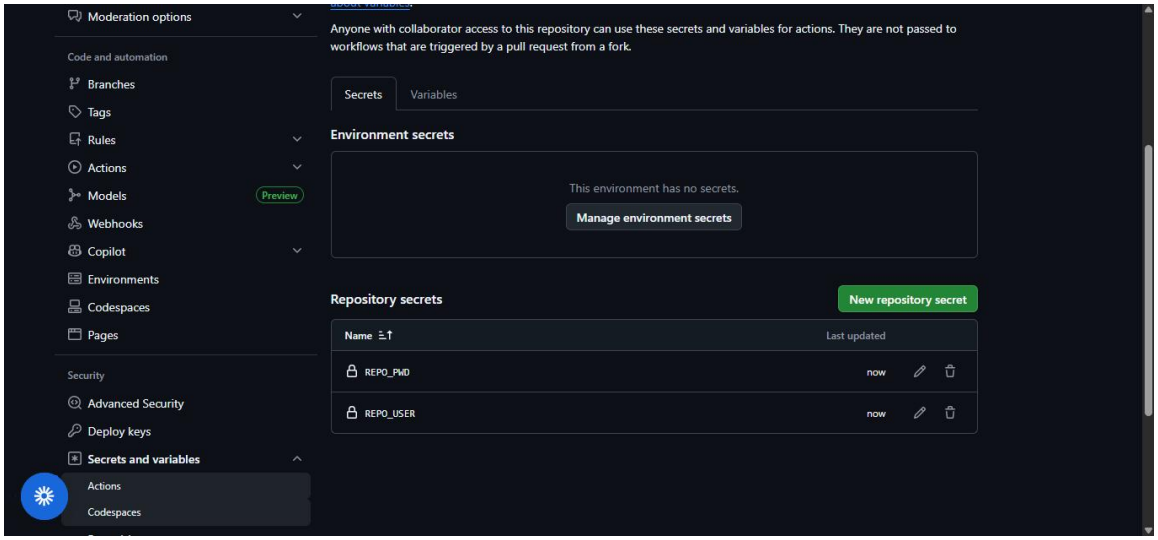
5. Building a Docker Image & Run Image Scan: I setup a job in the github action to install docker.



Setup the docker username and pasword so the action and call it to run the image scan.

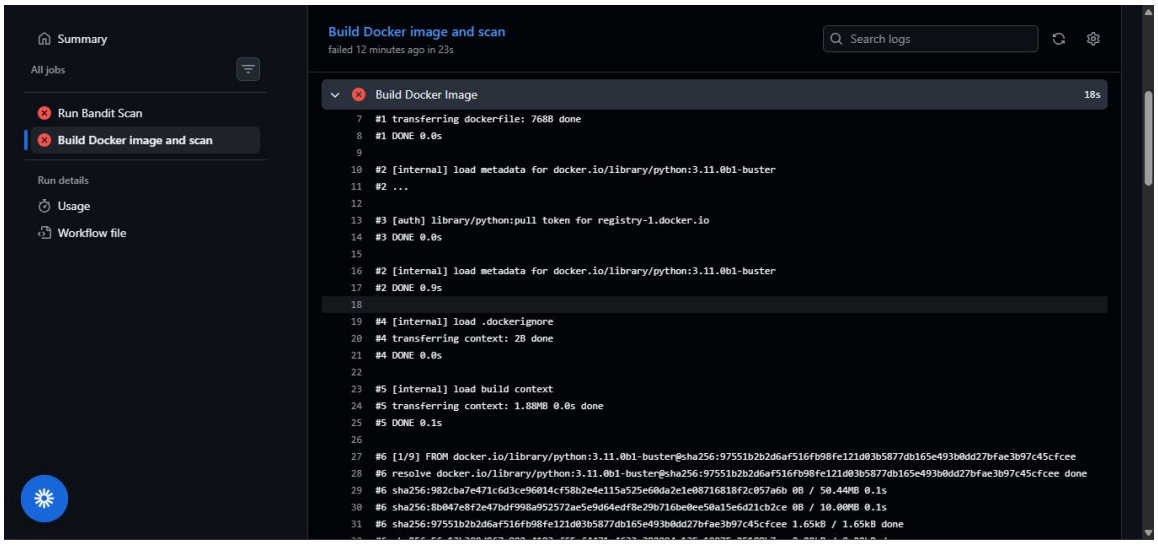


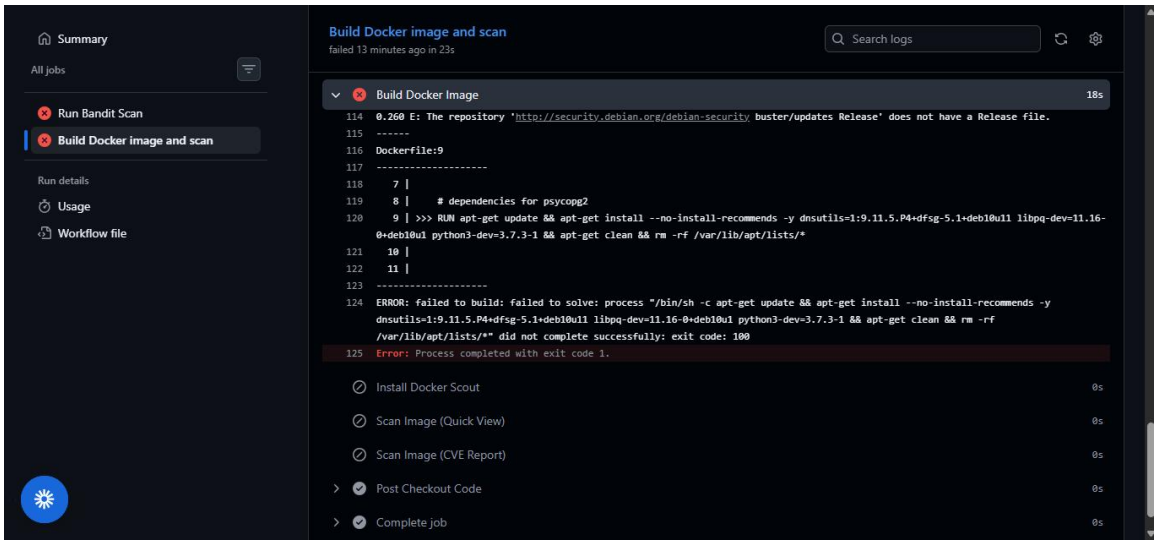
I went to settings and scroll, select actions and veriables to store my docker login details



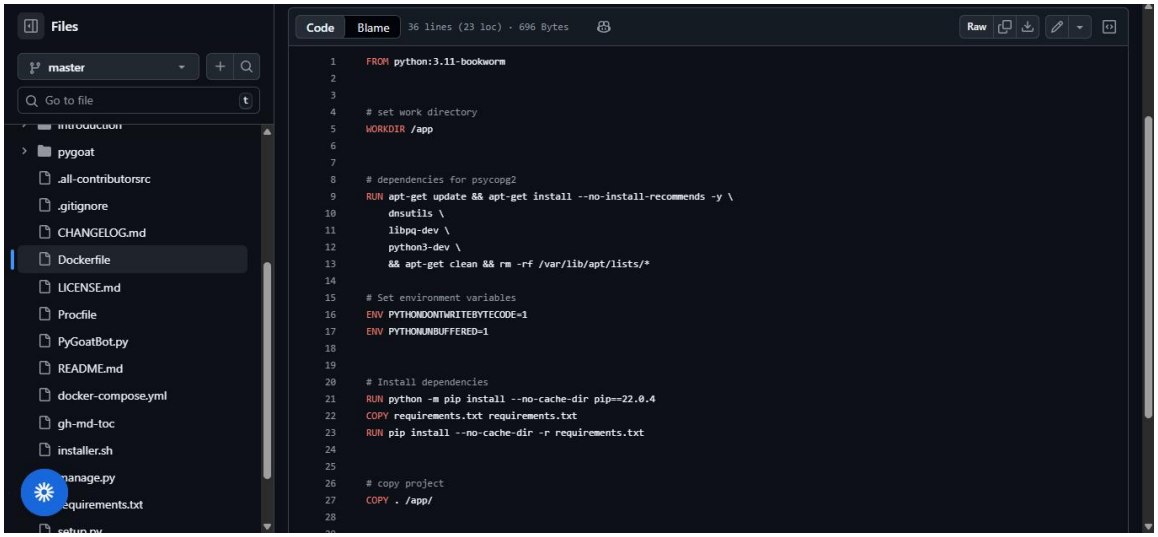
Docker username and password stored, this is to prevent exposure of private credentials in the repository.

Commit changes and see results





Got an error because the python library (python:3.11.0b1-buster) used in the pygoat web app stopped working 2024 so I updated the docker file in the github repo so the the image can be built properly



I changed the library to (python:3.11-bookworm) as it is the updated library, commit changes and the Security CI pipeline ran automatically

I used ‘docker scout quickview’ to get an overview of the images dependencies just as seen in the screenshot below.

Summary

All jobs

Run Bandit Scan

Build Docker image and scan

Run details

Usage

Workflow file

Build Docker image and scan

succeeded now in 2m 25s

Search logs

Build Docker Image 51s

```
#13 1.362   Applying socialaccount.0001_initial... OK
#13 1.392   Applying socialaccount.0002_token_max_lengths... OK
#13 1.413   Applying socialaccount.0003_extra_data_default_dict... OK
#13 DONE 1.6s
#14 [9/9] WORKDIR /app
#14 DONE 0.0s
#15 exporting to image
#15 exporting layers
#15 exporting layers 4.7s done
#15 writing image sha256:5a28b16160252a1ff42547097c8b192304bd32a81aced1f3476421fec2b81ffd done
#15 naming to docker.io/library/myapp:latest done
#15 DONE 4.7s
```

Install Docker Scout 4s

```
▶ Run curl -fsSL https://raw.githubusercontent.com/docker/scout-cli/main/install.sh -o install-scout.sh

[info] fetching release script for tag='v1.20.3'
[info] using release tag='v1.20.3' version='1.20.3' os='linux' arch='amd64'
[info] installed /home/runner/.docker/cli-plugins/docker-scout
Login Succeeded
WARNING! Your credentials are stored unencrypted in '/home/runner/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
```

Summary

All jobs

Run Bandit Scan

Build Docker image and scan

Run details

Usage

Workflow file

Build Docker image and scan

succeeded now in 2m 25s

Search logs

Scan Image (Quick View) 1m 24s

```
▶ Run docker scout quickview myapp:latest

   ...Storing image for indexing
   ✓ Image stored for indexing
   ...Indexing
   ✓ Indexed 649 packages
   i Base image was auto-detected. To get more accurate results, build images with max-mode provenance attestations.
     Review https://docs.docker.com/build/attestations/slsa-provenance/ for more information.
 Target             |  myapp:latest          |    7C    40H    59M   147L     7?
   digest           |  5a28b1616025          |
 Base image         |  python:3.11-bookworm  |    0C     9H    20M   137L     5?
 Updated base image |  python:3.12-bookworm  |    0C     8H    20M   137L     5?
                    |                        |                  -1

What's next:
    View vulnerabilities → docker scout cves myapp:latest
    View base image update recommendations → docker scout recommendations myapp:latest
    Include policy results in your quickview by supplying an organization → docker scout quickview myapp:latest --org
<organization>
```

Scan Image (CVE Report) 2s

```
▶ Run docker scout cves myapp:latest

   ✓ SBOM of image already cached, 649 packages indexed
```

Summary

All jobs

Run Bandit Scan

Build Docker image and scan

Run details

Usage

Workflow file

Build Docker image and scan

succeeded 7 minutes ago in 2m 25s

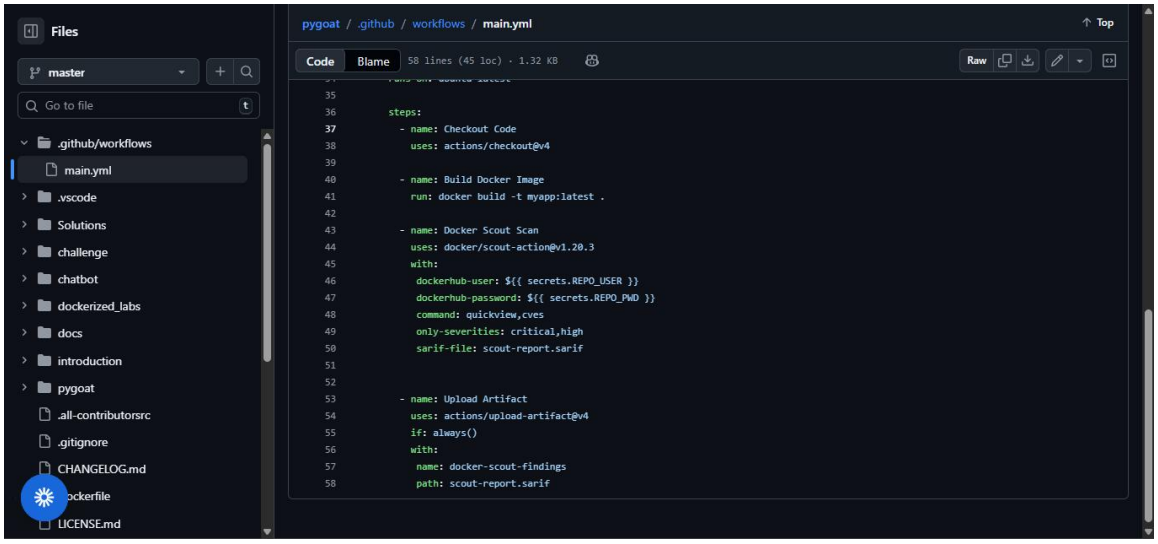
Search logs

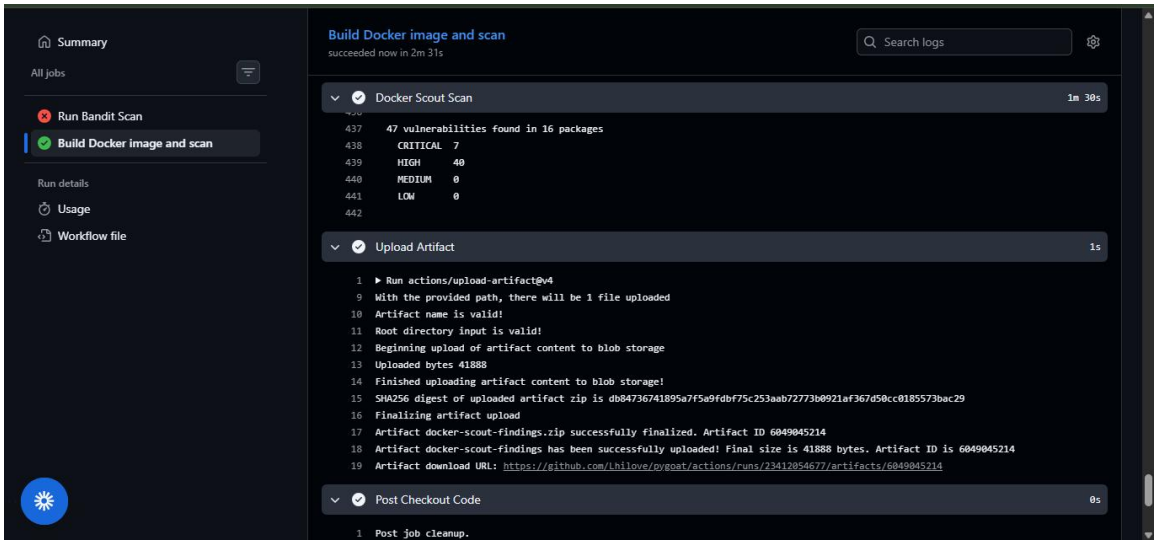
Scan Image (CVE Report) 2s

```
      Fixed version  : not fixed

    X LOW CVE-2025-69646
      https://scout.docker.com/v/CVE-2025-69646?s=debian&n=binutils&ns=debian&t=deb&osn=debian&osv=12&vr=%3C%3D2.40-2
      Affected range : <=2.40-2
      Fixed version  : not fixed

    X LOW CVE-2025-69645
      https://scout.docker.com/v/CVE-2025-69645?s=debian&n=binutils&ns=debian&t=deb&osn=debian&osv=12&vr=%3C%3D2.40-2
      Affected range : <=2.40-2
      Fixed version  : not fixed

    X LOW CVE-2025-69644
      https://scout.docker.com/v/CVE-2025-69644?s=debian&n=binutils&ns=debian&t=deb&osn=debian&osv=12&vr=%3C%3D2.40-2
      Affected range : <=2.40-2
      Fixed version  : not fixed

    X LOW CVE-2025-66866
      https://scout.docker.com/v/CVE-2025-66866?s=debian&n=binutils&ns=debian&t=deb&osn=debian&osv=12&vr=%3C%3D2.40-2
      Affected range : <=2.40-2
      Fixed version  : not fixed

    X LOW CVE-2025-66865
      https://scout.docker.com/v/CVE-2025-66865?s=debian&n=binutils&ns=debian&t=deb&osn=debian&osv=12&vr=%3C%3D2.40-2
      Affected range : <=2.40-2
      Fixed version  : not fixed
```

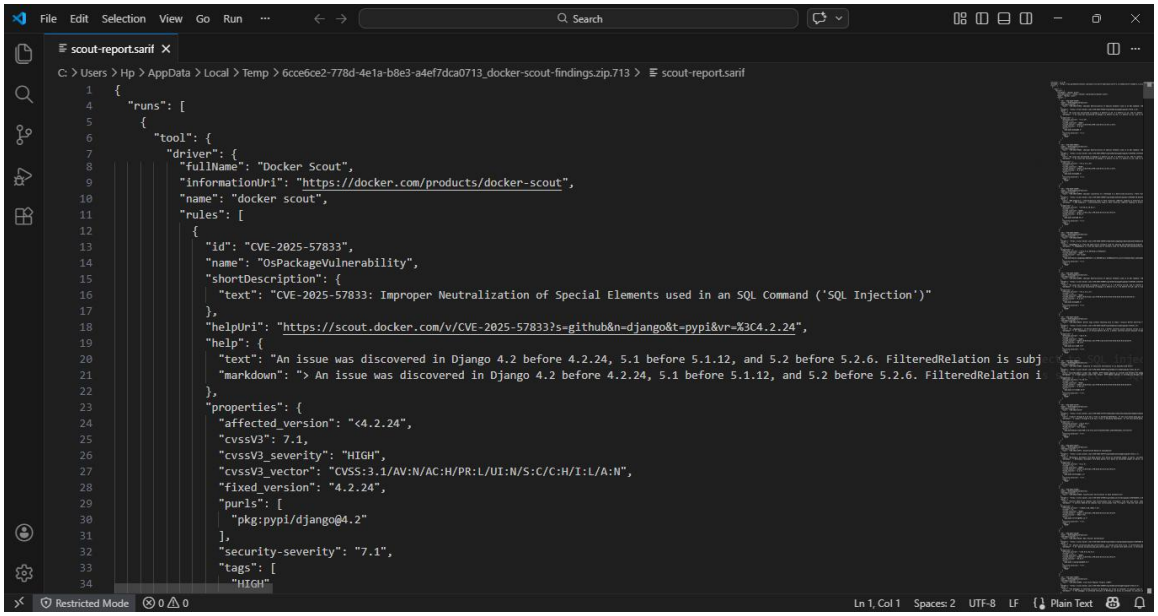
https://scout.docker.com/v/CVE-2025-66866?s=debian&n=binutils&ns=debian&t...

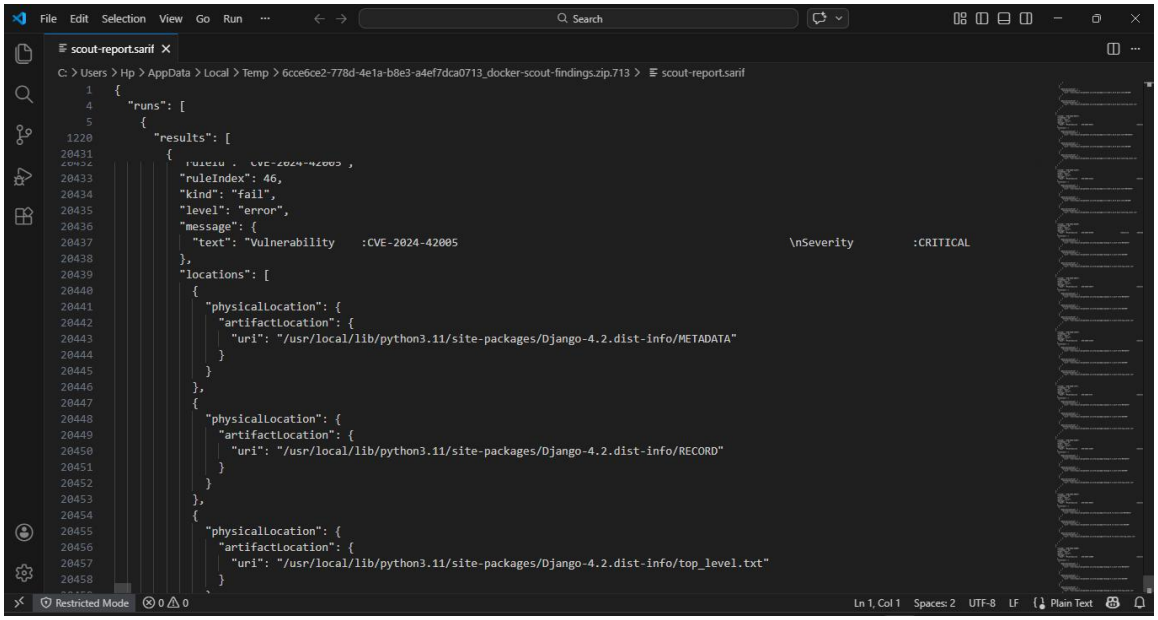
I have to ensure docker scout only scans for medium and high severity just to get the developers focused on the important vulnerabilities., also upload the reports as well.





The link to download the image scan report is in the artifact section and it is in sarif file format.





This is the report which indicates, medium, high and critical serverity levels.

Conclusion:

This project doesn't show remediation of these vulnerabilites just how we can automate security into a software development life cycle (SSDLC).